

Programmation orientée objet en Python

- Chapitre 4:
- Listes et Boucles

Listes

- Une liste comme une collection d'éléments séparés par des virgules, l'ensemble étant enfermé dans des crochets

```
>>> jour = ['lundi', 'mardi', 'mercredi', 2020, 18.5, 'jeudi', 'vendredi']
>>> jour
['lundi', 'mardi', 'mercredi', 2020, 18.5, 'jeudi', 'vendredi']
>>> print (jour)
['lundi', 'mardi', 'mercredi', 2020, 18.5, 'jeudi', 'vendredi']
>>> █
```

- il est possible de changer les éléments individuels d'une

```
>>> print (jour)
['lundi', 'mardi', 'mercredi', 2020, 18.5, 'jeudi', 'vendredi']
>>> jour[3] = jour[3] + 37
>>> jour
['lundi', 'mardi', 'mercredi', 2057, 18.5, 'jeudi', 'vendredi']
>>> █
```

- Fonction len()

- >>> print(len(jour))
- 7

Listes

■ Fonction del

```
[>>> del(jour[4])  
[>>> jour  
['lundi', 'mardi', 'mercredi', 2057, 'jeudi', 'vendredi']  
>>> █
```

■ Fonction append()

```
[>>> jour.append('samedi')  
[>>> jour  
['lundi', 'mardi', 'mercredi', 2057, 'jeudi', 'vendredi', 'samedi']  
>>> █
```



Accès aux éléments

■ Soit les listes

- `>>> nombres = [15, 48, 20, 35]`
`>>> animaux = ["éléphant", "girafe", "rhinocéros", "gazelle"]`
- `>>> liste3 = [2000, "Bambey", 3.14, ["Ablaye", "Bambey", 1000]]`

<code>>>> print(nombres[2])</code>	<code>>>> print(nombres[1:3])</code>	<code>>>> print(nombres[2:3])</code>	<code>>>> print(nombres[2:])</code>	<code>>>> print(nombres[:2])</code>	<code>>>> print(nombres[-1])</code>	<code>>>> print(nombres[-2])</code>
20	[48, 20]	[20]	[20, 35]	[15, 48]	35	20

■ Notation

- la notation `liste[i:j]` désigne les éléments `i` à `j-1` de la liste
- la notation `[i:]` désigne les éléments `i` et suivants
- la notation `[:i]` désigne les éléments 0 à `i-1`

Les listes sont modifiables

■ Modification par affectation directe

■ Exemple1:

- `>>> nombres[0] = 12`
- `>>> nombres`
- `[12, 48, 20, 35]`

■ Exemple2:

- `>>> liste3[3][1] = "Dahirou"`
 `>>> liste3`
- `[2000, "Bambey", 3.14, ["Ablaye", "Dahirou", 1980]]`

■ Notation

- la notation `liste[i:j]` désigne les éléments `i` à `j-1` de la liste
- la notation `[i:]` désigne les éléments `i` et suivants
- la notation `[:i]` désigne les éléments 0 à `i-1`

Les listes sont modifiables

■ Les listes peuvent être modifiées

- Ajouter un élément à la fin
- Ajouter un élément à une position donnée
- Enlever un élément
- Enlever un élément à une position donnée

• Exemples

```
animaux = ["éléphant", "girafe",  
           "rhinocéros", "gazelle"]
```

```
animaux.append("lion")
```

```
animaux.insert(0, "hippopotame")
```

```
animaux.remove("gazelle")
```

```
del animaux[-2]
```

Que contient alors la liste animaux ?

```
["hippopotame", "éléphant", "girafe", "lion"]
```

Les listes sont modifiables

- Ajouter tous les éléments d'une autre liste
 - Trier une liste (ordre numérique / alphabétique)
 - Concaténer deux listes
 - Inverser l'ordre d'une liste
 - Cloner une liste
- Exemples
 - `animaux.extend(["lion", "buffle"])`
 - `animaux.sort()`
 - `animaux + ["lion", "buffle"]`
 - `animaux.reverse()`
 - `list(animaux)`

Listes (Exemples (1))

■ Supprimer une entrée avec son index avec la fonction **del**

```
>>> liste = ["a", "b", "c"]
```

```
>>> del liste[1]
```

```
>>> liste
```

```
['a', 'c']
```

□ Supprimer une entrée avec sa valeur avec la méthode **remove**

```
>>> liste = ["a", "b", "c"]
```

```
>>> liste.remove("a")
```

```
>>> liste
```

```
['b', 'c']
```

Listes (Exemples (2))

❑ Inverser les valeurs d'une liste avec la méthode **reverse**

```
>>> liste = ["a", "b", "c"]
```

```
>>> liste.reverse()
```

```
>>> liste
```

```
['c', 'b', 'a']
```

❑ Compter le nombre d'items d'une liste

```
>>> liste = [1,2,3,5,10]
```

```
>>> len(liste)
```

```
5
```

Listes (Exemples (3))

❑ Compter le nombre d'occurrences d'une valeur

```
>>> liste = ["a","a","a","b","c","c"]
```

```
>>> liste.count("a")
```

3

```
>>> liste.count("c")
```

2

❑ Trouver l'index d'une valeur

```
>>> liste = ["a","a","a","b","c","c"]
```

```
>>> liste.index("b")
```

3

Listes (Exemples (4))

❑ Trouver un item dans une liste avec le mot clé **in**

```
>>> liste = [1,2,3,5,10]
```

```
>>> 3 in liste
```

```
True
```

```
>>> 11 in liste
```

```
False
```

❑ La fonction range

```
>>> range(10)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Listes (Exemples (5))

❑ Agrandir une liste

```
>>> x = [1, 2, 3, 4]
```

```
>>> y = [4, 5, 1, 0]
```

```
>>> x.extend(y)
```

```
>>> print x
```

```
[1, 2, 3, 4, 4, 5, 1, 0]
```

❑ Additionner deux listes

```
>>> x = [1, 2, 3]
```

```
>>> y = [4, 5, 6]
```

```
>>> x + y
```

```
[1, 2, 3, 4, 5, 6]
```

Listes (Exemples (6))

❑ Multiplier une liste:

```
>>> x = [1, 2]
```

```
>>> x*5
```

```
[1, 2, 1, 2, 1, 2, 1, 2, 1, 2]
```

❑ Initialiser une liste:

- ```
>>> [0] * 5
```

- ```
[0, 0, 0, 0, 0]
```

Listes (Exemples (7))

❑ Obtenir le plus grand élément d'une liste, ou le plus petit :

- `>>> max([2, 1, 4, 3])`
- 4
- `>>> min([2, 1, 4, 3])`
- 1

❑ Faire la somme d'une liste:

- `>>> sum([2, 1, 4, 3])`
- 10

Listes et chaînes de caractères

- Couper une chaîne de caractères en une liste de chaînes : **split**

✗ **split sans argument** : couper sur les espaces, sauts de ligne, tabulations

```
>>> "Une phrase avec des mots".split(" ")
```

```
["Une", "phrase", "avec", "des", "mots"]
```

✗ **Split avec argument** : couper sur un séparateur donné

```
>>> "Plan-Jaxaay".split("-")
```

```
["Plan", "Jaxaay"]
```

- Assembler une liste de chaînes en une seule chaîne : **join**

```
>>> "/" .join(["répertoire", "sous_répertoire", "fichier"])
```

```
"répertoire/sous_répertoire/fichier"
```

Boucles

- Une boucle permet d'exécuter plusieurs fois les mêmes commandes

- La boucle parcourt une séquence

for *variable* **in** *sequence* :

commande répétée plusieurs fois

commande répétée plusieurs fois

suite du programme (exécuté une seule fois)

Boucles

- **La séquence peut être :**

- ✗ **une variable**

- 🟠 **Chaîne de caractères**

- 🟠 **Liste, ensemble, tuple**

- 🟠 **...**

- ✗ **une liste d'indices générée avec la fonction range()**

- range(5) => [0, 1, 2, 3, 4]**

Boucles

- Exemple de séquence de type chaîne de caractères:

```
[>>> prenom="Dahirou"  
[>>> for car in prenom:  
[...     print(car+' ',end=' ')  
[...  
D * a * h * i * r * o * u * >>> █
```

- L'instruction **for** permet donc d'écrire des boucles, dans lesquelles l'itération traite successivement tous les éléments d'une séquence donnée.
- L'argument **end = " "** signifie que vous souhaitez remplacer le saut à la ligne par un simple espace. Si vous supprimez cet argument, les nombres seront affichés les uns en-dessous des autres.

Boucles

■ Exemple de séquence de type Liste

```
[>>> liste=['SATIC','SDD','ECOMIJ']
[>>> for ufr in liste:
[...     print('longueur de la chaine',ufr,'=',len(ufr))
[...
longueur de la chaine SATIC = 5
longueur de la chaine SDD = 3
longueur de la chaine ECOMIJ = 6
>>> █
```

```
environnements = ["savane", "forêt", "savane", "rivière"]
```

✗ Afficher un animal par ligne

```
for animal in animaux : print(animal)
print("C'est fini !")
```

✗ Afficher un animal par ligne avec son numéro

```
for i in range(len(animaux)) :
    print("Numéro " + str(i) + " : " + animaux[i])
print("C'est fini !")
```

✗ Afficher un animal par ligne avec son environnement

```
for i in range(len(animaux)) :
    print(animaux[i] + " dans la " + environnements[i])
```

Boucles (boucler sur plusieurs listes avec zip)

■ Exemples

```
animaux = ["éléphant", "biche", "rhinocéros", "brochet"]  
environnements = ["savane", "forêt", "savane", "rivière"]
```

- for animal, enviro in zip(animaux, environnements):
- print(animal + " vit dans " + enviro)

■ Ce qui donne :

- 'éléphant vit dans savane
- biche vit dans forêt
- Rhinocéros vit dans savane
- brochet vit dans rivière



Boucles (boucler sur plusieurs listes avec des boucles imbriquées)

■ Exemples

```
animaux = ["éléphant", "biche", "rhinocéros", "brochet"]  
environnements = ["savane", "forêt", "savane", "rivière"]
```

- for animal in animaux:
- for enviro in environnements:
- Print (animal+ " dans " + enviro)

■ Ce qui donne :

- éléphant dans savane'
- éléphant dans forêt
- éléphant dans rivière
- biche dans savane
biche dans forêt
biche dans rivière

rhinocéros dans savane
rhinocéros dans forêt
rhinocéros dans rivière
brochet dans savane
brochet dans forêt
brochet dans rivière

Boucles

- **break** permet d'interrompre la boucle
- **continue** permet de passer immédiatement à l'élément suivant
- La boucle for peut aussi avoir un bloc **else** :
 - for** *variable* **in** *liste* :
 - if** *condition d'exclusion* :
 - continue**
 - commande répétée plusieurs fois*
 - if** *condition* :
 - commande exécutée si condition satisfaite*
 - break**
 - else** :
 - commande exécutée si condition jamais satisfaite*

Boucles

■ **continue** : permet de passer immédiatement à l'élément suivant

for *variable* **in** *liste* :

if *condition d'exclusion* : **continue**

commande répétée plusieurs fois

Boucles

■ **break** et **else** :

✗ **break** permet d'interrompre la boucle

✗ le bloc **else** est exécuté si la boucle est allée jusqu'au bout (sans rencontrer **break**)

for *variable in liste* :

commande répétée plusieurs fois

if *condition* :

commande exécutée si condition satisfaite

break

else :

commande exécutée si condition jamais satisfaite

Boucles

- La boucle **while** dure tant qu'une condition est satisfaite :
while condition :

commande répétée plusieurs fois

suite du programme (la boucle est finie)

- Elle peut aussi avoir un **break** et un **else**
- Exemple: la liste des carrés et des cubes des nombres de

```
1 1² 1³  
[>>> a=0  
[>>> while a<12:  
[...     a=a+1  
[...     print(a , a**2 , a**3)  
[...     
```

- Afficher les dix premiers termes d'une suite appelée « Suite de Fibonacci »

Boucles

- Afficher les dix premiers termes d'une suite appelée « Suite de Fibonacci ». Il s'agit d'une suite de nombres dont chaque terme est égal à la somme des deux termes

```
[>>> a, b, c = 1, 1, 1  
[>>> while c < 11 :  
[...     print(b, end = " ")  
[...     a, b, c = b, a+b, c+1
```

```
l...  
1 2 3 5 8 13 21 34 55 89 >>> █
```

■ **resultat**

Exercice 2: Nombres paires d'une liste

■ Exemple2 : Garder seulement les nombres pairs d'une liste :

- `nombres = [1, 4, 7, 8, 12, 15]`
- `nombres_pairs = [ ] ...????`

Listes procédurales (*comprehension list*)

- Permet de créer une liste sans donner les éléments un à un mais en donnant une boucle pour la remplir

```
>>> [len(mot) for mot in "Un texte avec des mots".split()]
```

```
[2, 5, 4, 3, 4]
```

`sum()` retourne la somme d'une liste

```
>>> sum(len(mot) for mot in "Un texte avec des mots".split())
```

```
18
```

- Plusieurs boucles peuvent être combinées

```
print(" " + seq1)
```

```
print("\n".join(base2 + "".join([" ", "X"][base1 == base2] for base1 in  
                                seq1) for base2 in seq2 ))
```

- => Plusieurs approches différentes pour faire la même chose
✗ Mais en général, une approche est plus simple et claire !